Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №1

«Локальные скрипты»

Выполнили:

Матвеев Илья Александрович, студент группы N3351

_____
(подпись)

Проверил:

Ищенко Алексей Петрович, преподаватель ФБИТ, программист 1 категории

_____
(отметка о выполнении)

_____
(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Содержание 2
Введение 3
1 Ход работы 4
1.1 Локальная программа 4
1.1.1 Инструменты для разработки 4
1.1.2 Описание функций 4
1.1.3 Итоговый функционал приложения 5
1.1.4 Исходный код приложения 6
1.1.5 Тестирование приложения 9
1.2 Веб-скрипт 11
1.2.1 Инструменты для разработки 12
1.2.2 Описание функций, реализующих ограничения 12
1.2.3 Итоговый функционал скрипта 13
1.2.4 Исходный код скрипта 14
1.2.5 Тестирование скрипта 17
Заключение 20

ВВЕДЕНИЕ

Цель работы – изучение создания локальных приложений с графическим интерфейсом и улучшение навыков создания веб-скриптов.

Для достижения поставленной цели необходимо решить следующие задачи:

- Проанализировать поставленные задания, выбрать оптимальный подход и язык программирования;
- Реализовать первую часть задания – локальная программа, контролирующая доступ к файлам;
- Реализовать вторую часть задания – веб-скрипт, моделирующий запрет доступа к содержимому.

1 ХОД РАБОТЫ

В данном разделе представлен ход выполнения задач, поставленных во введении. В первом разделе реализована локальная программа для Linux, во втором разделе – указанный веб скрипт.

1.1 Локальная программа

Задание устанавливается следующим образом:
Необходимо разработать программу, запрещающую в текущем (том, котором она находится) каталоге создание, копирование, удаление или переименование файлов с заданными именами (можно использовать маски файлов). Список имен или их шаблонов хранить в файле template.tbl, как текст. Должна быть обеспечена защита этого файла от удаления, несанкционированного просмотра и модификации. При установке программы можно предусмотреть ее отключение с использованием пароля, хранящегося в первой строке файла template.tbl в хешированном виде. Программа должна включать и выключать режим защиты.

1.1.1 Инструменты для разработки

В качестве основного инструмента для разработки был выбран язык программирования Python и операционная система Linux – как наиболее подходящие для поставленного задания вещи. В Python использовался набор библиотек:

- PyQt6 – универсальная библиотека для создания графического интерфейса пользователя при помощи сигналов – вокруг нее построены основные окна приложения и в целом вся визуальная часть;
- Встроенная библиотека pathlib – для получения текущего местонахождения файла;
- Встроенные библиотеки sys, os – для работы с операционной системой;
- Встроенные библиотеки subrocess, traceback, hashlib – для обработки текущего процесса, обработки ошибок и хэширования пароля в файл.

1.1.2 Описание функций

В программе был использован широкий набор функций. Все функции представлены в таблице 1.

Таблица 1 – Используемые функции

Функция	Назначение
get_app_dir()	Возвращает путь к каталогу, где находится исполняемый файл программы или исходный .py. Используется для определения расположения файлов защиты.
get_directory()	Обёртка над get_app_dir() — возвращает директорию приложения для операций с файлами защиты.
create_template_file()	Создаёт template.tbl, если он отсутствует: записывает хеш пароля в первую строку и далее — список защищаемых файлов/масок.
protect_file()	Применяет защиту к указанным файлам: устанавливает атрибут immutable (chattr +i) и убирает права доступа (chmod 000), запрещая изменение, удаление и переименование.
unprotect_file()	Снимает защиту: удаляет атрибут immutable (chattr -i) и восстанавливает обычные права доступа (chmod 644).
open_template_file()	Открывает файл template.tbl через системный редактор (xdg-open) для просмотра содержимого.
toggle_protection()	Переключает состояние защиты: если активна — запрашивает пароль; если выключена — включает.
enable_protection()	Включает защиту файлов (без ввода пароля). Вызывает protect_file().
disable_protection()	Отключает защиту (после ввода пароля). Вызывает unprotect_file().

1.1.3 Итоговый функционал приложения

Данное приложение реализует весь функционал, описанный в задании. Оно умеет по шаблону определять, к каким файлам запретить пользовательский доступ – и запрещает копирование, удаление, изменение и любые действия над файлами, имена которых заданы в шаблоне. Кроме того, внутри шаблона так же содержится пароль для разблокировки

доступа к файлам. Кроме этого, из приложения можно открыть файл шаблона и выключить защиту.

1.1.4 Исходный код приложения

В листинге 1 представлен исходный код приложения с комментариями.

Листинг 1 – Исходный код приложения

```
from PyQt6.QtWidgets import (QApplication, QWidget, QVBoxLayout, QLabel,
QPushButton, QLineEdit)
from PyQt6.QtCore import Qt
from PyQt6 import QtWidgets
import sys
from pathlib import Path
import hashlib
import subprocess
import traceback
import os

APP_NAME = "Lab1Matveev"
TEMPLATE_FILENAME = "template.tbl"
PASSWORD = "key1234"

class MainWindow(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Основное окно")
        self.resize(600, 600)
        layout = QVBoxLayout()
        self.protection_enabled = True

        self.info = QLabel(
            "Данная программа запрещает редактирование, удаление, "
            "создание, копирование или переименование файлов с заданными
именами.\n\n"
            "Маски названий файлов находятся в template.tbl"
        )
        self.info.setWordWrap(True)

        self.btn_toggle = QPushButton("Отключить режим защиты")
        self.btn_open = QPushButton("Открыть файл template.tbl")

        layout.addWidget(self.info)
        layout.addWidget(self.btn_toggle)
        layout.addWidget(self.btn_open)
        self.setLayout(layout)

        self.btn_toggle.clicked.connect(self.toggle_protection)
        self.btn_open.clicked.connect(self.open_template_file)

    def open_password_window(self):
        self.pass_window = PasswordWindow(self.disable_protection)
        self.pass_window.show()

    def toggle_protection(self):
        if self.protection_enabled:
            self.open_password_window()
        else:
```

```
            self.enable_protection()

    def disable_protection(self):
        self.protection_enabled = False
        unprotect_file()
        self.btn_toggle.setText("Включить режим защиты")

    def enable_protection(self):
        self.protection_enabled = True
        protect_file()
        self.btn_toggle.setText("Отключить режим защиты")

    def open_template_file(self):
        file_path = get_directory() / "template.tbl"
        try:
            subprocess.Popen(["xdg-open", str(file_path)])
        except Exception:
            traceback.print_exc()

class PasswordWindow(QWidget):
    def __init__(self, on_success_callback):
        super().__init__()
        self.setWindowTitle("Ввод пароля")
        self.resize(300, 150)
        self.on_success_callback = on_success_callback

        layout = QVBoxLayout()
        self.label = QLabel("Введите пароль:")
        self.input = QLineEdit()
        self.input.setEchoMode(QLineEdit.EchoMode.Password)
        self.btn_ok = QPushButton("Подтвердить")
        self.btn_ok.clicked.connect(self.check_password)

        layout.addWidget(self.label)
        layout.addWidget(self.input)
        layout.addWidget(self.btn_ok)
        self.setLayout(layout)

    def check_password(self):
        if self.input.text() == PASSWORD:
            self.on_success_callback()
            self.close()
        else:
            self.label.setText("Неверный пароль, попробуйте снова.")

class InternalErrorWindow(QWidget):
    def __init__(self, message: str):
        super().__init__()
        self.setWindowTitle("Ошибка приложения")
        self.resize(500, 250)
        layout = QVBoxLayout(self)
        layout.addWidget(QLabel(
            "<b>Проблема во внутренней работе программы.</b><br>"
            "Перезапустите приложение или обратитесь к разработчику."
        ))
        btn = QPushButton("Закрыть")
        btn.clicked.connect(QtWidgets.QApplication.instance().quit)
        layout.addWidget(btn)

class AdminRightsWindow(QWidget):
    def __init__(self):
        super().__init__()
```

```
        self.setWindowTitle("Ошибка доступа")
        self.resize(400, 150)
        layout = QVBoxLayout()
        label = QLabel("Приложение необходимо запустить с правами root
(sudo).")
        label.setWordWrap(True)
        label.setAlignment(Qt.AlignmentFlag.AlignCenter)
        btn_ok = QPushButton("OK")
        btn_ok.clicked.connect(self.close)
        layout.addWidget(label)
        layout.addWidget(btn_ok, alignment=Qt.AlignmentFlag.AlignCenter)
        self.setLayout(layout)

def get_app_dir() -> Path:
    if getattr(sys, "frozen", False):
        return Path(sys.executable).parent
    return Path(__file__).parent

def create_template_file():
    app_dir = get_app_dir()
    file_path = app_dir / TEMPLATE_FILENAME
    if not file_path.exists():
        with open(file_path, "w", encoding="utf-8") as f:
            hex_dig = hashlib.sha256(PASSWORD.encode()).hexdigest()
            f.write(hex_dig + "\n")
            for name in PROTECTED_FILES:
                f.write(name + "\n")

def get_directory() -> Path:
    return get_app_dir()

BASE_DIR = get_directory()
PROTECTED_FILES = ["important.txt", "key.txt", "secret.bin", "logs.txt",
"important_tbl.tbl"]

def protect_file():
    for filename in PROTECTED_FILES:
        temp_path = BASE_DIR / filename
        if temp_path.exists():
            try:
                subprocess.run(["chmod", "000", str(temp_path)], check=True)
                subprocess.run(["chattr", "+i", str(temp_path)], check=True)
            except subprocess.CalledProcessError:
                traceback.print_exc()

def unprotect_file():
    for filename in PROTECTED_FILES:
        temp_path = BASE_DIR / filename
        if temp_path.exists():
            try:
                subprocess.run(["chattr", "-i", str(temp_path)], check=True)
                subprocess.run(["chmod", "644", str(temp_path)], check=True)
            except subprocess.CalledProcessError:
                traceback.print_exc()

if __name__ == "__main__":
    if os.geteuid() != 0:  # Проверка sudo
        app = QApplication(sys.argv)
        window = AdminRightsWindow()
        window.show()
        sys.exit(app.exec())
    else:
```

```
try:
    create_template_file()
    protect_file()
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())
except Exception as e:
    traceback.print_exc()
    app = QApplication(sys.argv)
    error_window = InternalErrorWindow(str(e))
    error_window.show()
    app.exec()
```

1.1.5 Тестирование приложения

Сценарий для тестирования:

- Интерфейс приложения без прав администратора (рисунок 1);
- Каталог с файлами до запуска приложения (рисунок 2);
- Интерфейс программы (рисунок 3);
- Каталог после запуска программы (рисунок 4);
- Ошибка удаления при запущенной программе (рисунок 5);
- Экран ввода пароля для снятия защиты (рисунок 6);

Проведение тестирования представлено на обозначенных рисунках ниже.

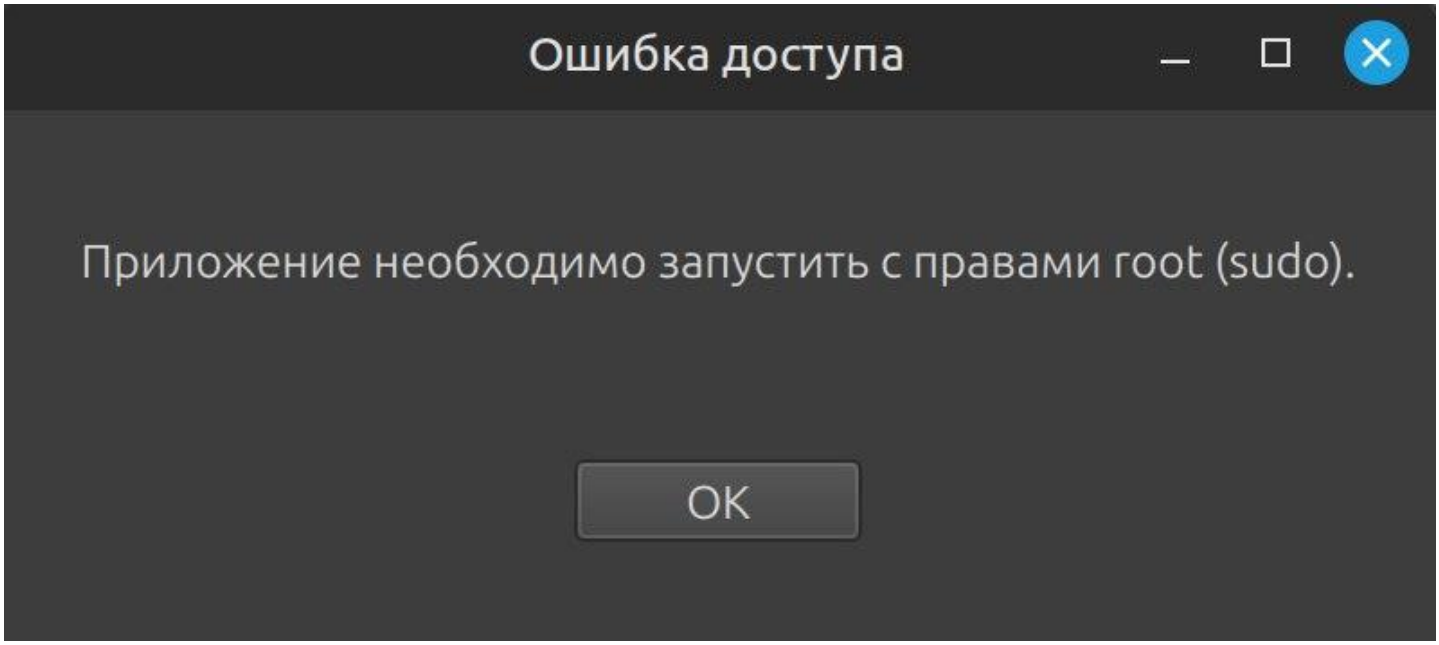


Рисунок 1 – Приложение без прав администратора

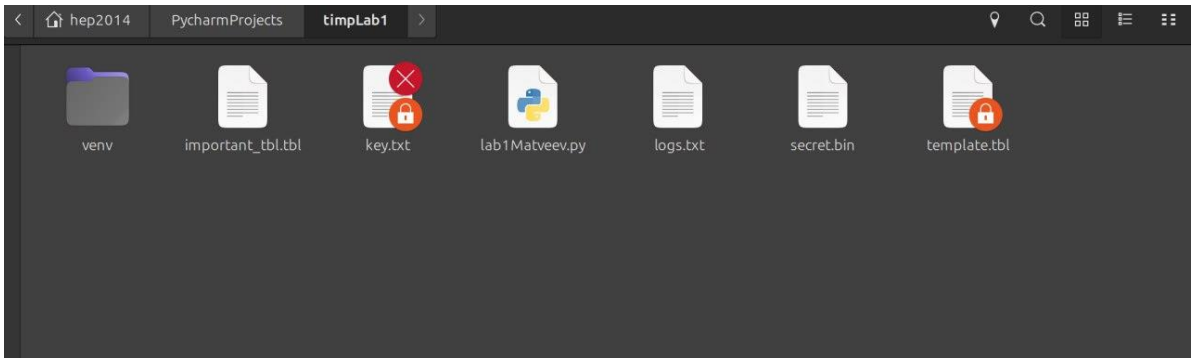


Рисунок 2 – Каталог до запуска приложения

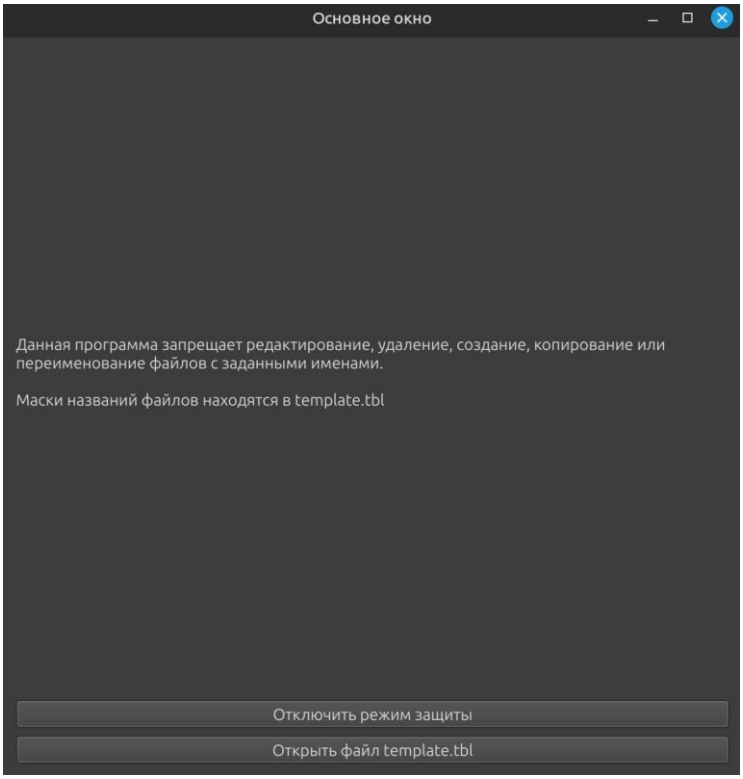


Рисунок 3 – Интерфейс программы

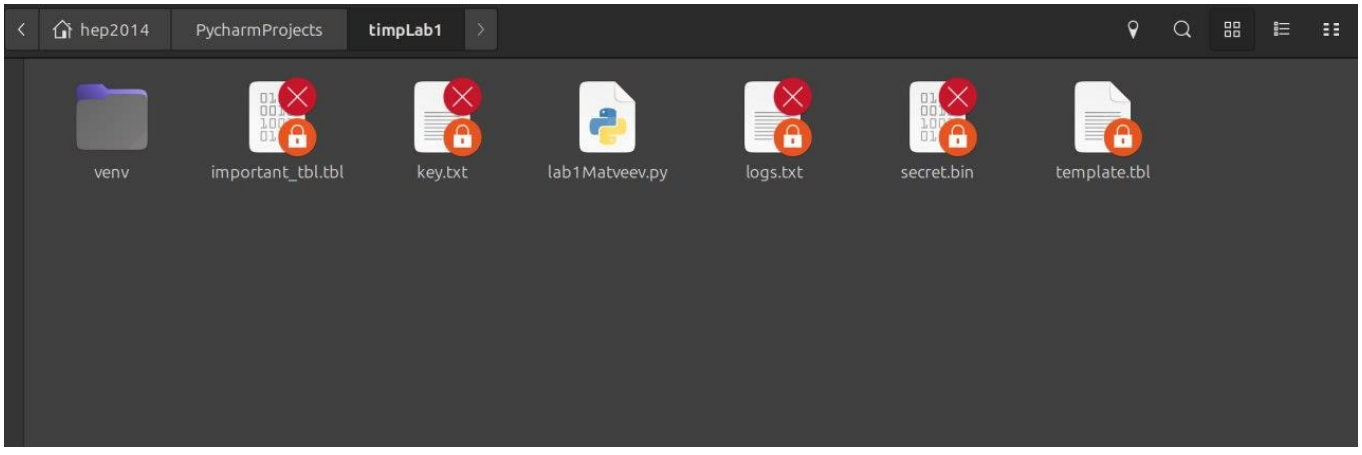


Рисунок 4 – Каталог после запуска программы

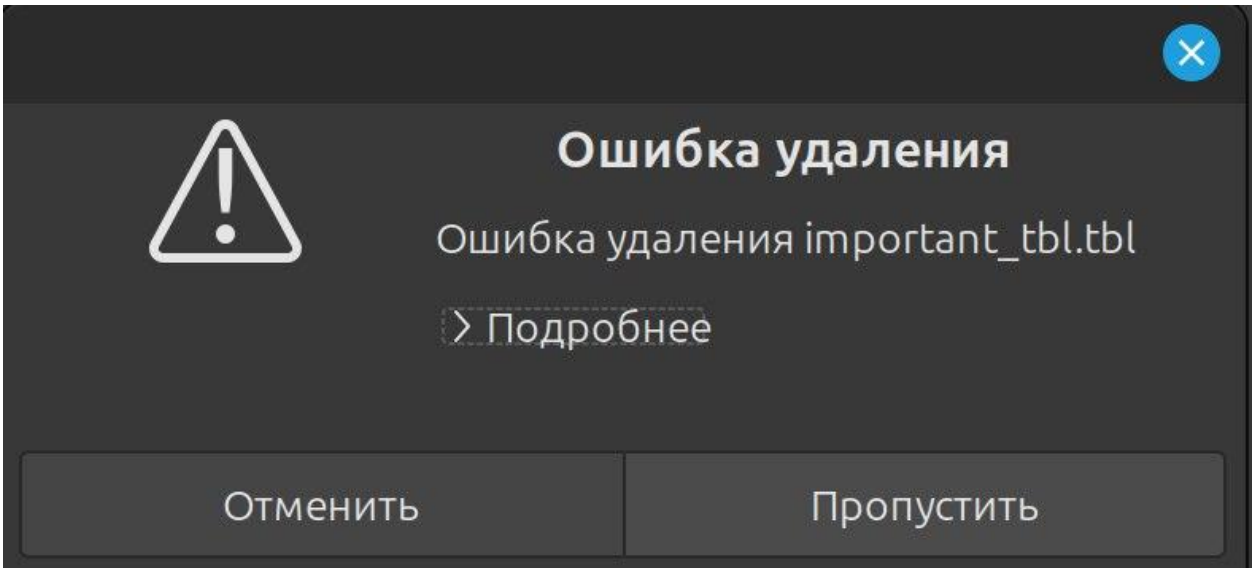


Рисунок 5 – Ошибка удаления

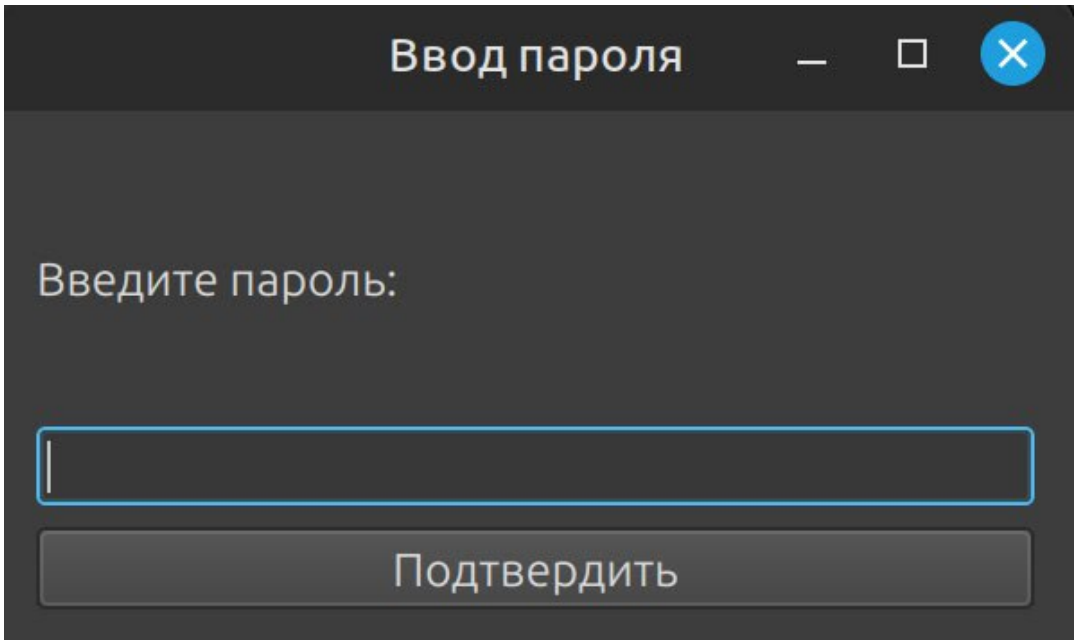


Рисунок 6 – Экран ввода пароля для снятия защиты

Как мы видим, все тесты подтвердили функционал приложения.

1.2 Веб-скрипт

Задание устанавливается следующим образом:

Реализовать скрипт, внедряемый в код HTML-документа, который реализует защиту от копирования в буфер (нет возможности выделять содержимое, копировать в буфер как текст или через скриншот и т.д.) и сохранения всех страниц, вызываемых с текущей. При этом печать этих страниц на бумажный носитель должна быть доступна. Отключение

скрипта должно происходить с использованием пароля, хранящегося в теле скрипта в зашифрованном виде.

1.2.1 Инструменты для разработки

Для разработки была выбрана эффективная комбинация средств, которая позволяет выполнить поставленную задачу. Для начала – основной используемый язык программирования – Python со встроенными библиотеками:

- Base64 – используется для хэширования ключа, позволяющего снять защиту;
- Textwrap.dedent – для формирования html-кода внутри Python.

Кроме этого, внутри скрипта на языке Python используется создание страницы при помощи HTML и скриптового языка JavaScript, который реализует поставленные ограничения.

1.2.2 Описание функций, реализующих ограничения

В программе используются функции, которые реализуют ограничения и внутреннюю работу. Их список представлен в таблице 2.

Таблица 2 – Используемые функции

Функция	Назначение
xorDecrypt(b64, key)	Выполняет расшифровку пароля, хранящегося в скрипте в виде Base64-строки. Использует обратную операцию XOR для восстановления исходного текста пароля. Применяется при проверке введённого пользователем пароля.
enableProtection()	Активирует защитный режим страницы: включает запрет выделения текста, блокирует контекстное меню, операции копирования и вырезания, перехватывает ряд сочетаний клавиш, добавляет параметр защиты ко всем ссылкам. Выполняется при загрузке страницы.
disableProtection()	Полностью отключает защиту путём клонирования элемента <body>, что удаляет все ранее установленные обработчики событий. После замены body повторно подключает обработчики механизма затемнения экрана.

Продолжение таблицы 2

keyHandler(e)	Обрабатывает нажатия клавиш и блокирует сочетания, позволяющие копировать, сохранять или просматривать структуру страницы (Ctrl+C, Ctrl+S, Ctrl+U и др.). Печать (Ctrl+P) не блокируется.
linkHandler(e)	Перехватывает переходы по ссылкам и автоматически добавляет к URL параметр protected=1, позволяющий сохранять защитный режим на последующих страницах.
blackoutScreen()	Создаёт чёрный оверлей поверх всего содержимого страницы при потере фокуса окна или сворачивании вкладки. Предназначена для сокрытия информации в моменты переключения пользователя на другие приложения.
restoreScreen()	Удаляет оверлей, созданный функцией blackoutScreen(), при возвращении фокуса на вкладку или окно браузера.
installBlackoutListeners()	Подключает обработчики событий visibilitychange, blur и focus, обеспечивая автоматическое затемнение страницы при потере фокуса и его снятие при возвращении. Гарантирует работу механизма blackout даже после отключения основной защиты.
askPasswordAndDisable()	Запрашивает у пользователя пароль для отключения защиты, расшифровывает сохранённый пароль, сравнивает значения и при совпадении вызывает disableProtection(). Привязана к сочетанию клавиш Ctrl+Shift+L.
blockEvent(e)	Универсальная функция, блокирующая нежелательные события (копирование, контекстное меню и др.). Прерывает действие через preventDefault() и stopPropagation().

1.2.3 Итоговый функционал скрипта

Функционал скрипта полностью соответствуют заданию. Внутри python-кода создается html-документ, внутри которого нельзя ни копировать содержимое, ни вырезать, ни удалять – только печатать. Кроме того, при потере фокуса на странице (имитация попытки сделать скриншот), страница затемняется, что мешает зафиксировать ее содержимое. Более того, защиту можно снять при особом сочетании клавиш, используя пароль пользователя.

1.2.4 Исходный код скрипта

В листинге 2 представлен исходный код скрипта с комментариями.

Листинг 2 – Исходный код

```
import base64
from textwrap import dedent

# Инструкции - пароль и xor-пароль для шифрования
PASSWORD = "itmoSecret" # пароль
XOR_KEY = "my_key" # xor-ключ

# Функция, которая шифрует пароль с использованием base64 шифрования
def xor_encrypt(text: str, key: str) -> str:
    data = text.encode("utf-8")
    key_bytes = key.encode("utf-8")
    res = bytearray()
    for i, b in enumerate(data):
        res.append(b ^ key_bytes[i % len(key_bytes)])
    return base64.b64encode(res).decode("ascii")

# В следующей строке хранится зашифрованный пароль
enc_password_b64 = xor_encrypt(PASSWORD, XOR_KEY)

# Далее формируется защищенный HTML документ с использованием JavaScript
внутри
html = dedent(f"""\
    <!doctype html>
    <html lang="ru">
    <head>
        <meta charset="utf-8">
        <title>Илья Матвеев</title>
        <style>
            body.no-select {{
                -webkit-user-select: none;
                -moz-user-select: none;
                -ms-user-select: none;
                user-select: none;
            }}
            @media print {{
                body {{
                    -webkit-user-select: text;
                    -moz-user-select: text;
                    -ms-user-select: text;
                    user-select: text;
                }}
            }}
            #blackout-overlay {{
                position: fixed;
                top: 0;
                left: 0;
                width: 100vw;
                height: 100vh;
                background: black;
                z-index: 999999;
                pointer-events: none;
            }}
        </style>
    </head>
    <body>
```

```
        <h1>Матвеев Илья защитил</h1>
        <p>
            Университет ИТМО — это ведущий российский вуз в области
информационных и фотонных технологий, расположенный в Санкт-Петербурге.
            Он получил статус национального исследовательского университета,
развивает направления, связанные с IT, искусственным интеллектом,
            робототехникой, фотоникой и науками о жизни. ИТМО известен своей
сильной командой по спортивному программированию
            (семикратный чемпион ICPC) и высоким качеством образования.
        </p>
        <p>Нажмите Ctrl+Shift+L, чтобы попытаться отключить защиту.</p>

        <script>
            const XOR_KEY = "{XOR_KEY}";
            const ENC_PWD_B64 = "{enc_password_b64}";

            function xorDecrypt(b64, key) {{
                const bytes = atob(b64);
                let res = "";
                for (let i = 0; i < bytes.length; i++) {{
                    const k = key.charCodeAt(i % key.length);
                    res += String.fromCharCode(bytes.charCodeAt(i) ^ k);
                }}
                return res;
            }}

            //
            // --- БЛОКИРОВКА ДЕЙСТВИЙ ---
            //

            function enableProtection() {{
                document.body.classList.add("no-select");

                document.addEventListener("contextmenu", blockEvent, true);
                document.addEventListener("copy", blockEvent, true);
                document.addEventListener("cut", blockEvent, true);
                document.addEventListener("keydown", keyHandler, true);
                document.addEventListener("click", linkHandler, true);
            }}

            function disableProtection() {{
                // Убираем все навешанные обработчики, клонируя body
                const oldBody = document.body;
                const newBody = oldBody.cloneNode(true);
                newBody.classList.remove("no-select");
                oldBody.parentNode.replaceChild(newBody, oldBody);

                // ВАЖНО: заново навешиваем blackout-логику, чтобы она не
исчезла
                installBlackoutListeners();
            }}

            function blockEvent(e) {{
                e.preventDefault();
                e.stopPropagation();
            }}

            function keyHandler(e) {{
                const key = e.key.toLowerCase();
                if ((e.ctrlKey || e.metaKey) && ["c", "x", "s",
"u"].includes(key)) {{
                    e.preventDefault();
```

```
            e.stopPropagation();
        }}
    }}

    function linkHandler(e) {{
        const a = e.target.closest("a[href]");
        if (!a) return;
        const url = new URL(a.href, window.location.href);
        url.searchParams.set("protected", "1");
        a.href = url.toString();
    }}

    //
    // --- ЧЁРНЫЙ ЭКРАН ПРИ ПОТЕРЕ ФОКУСА ---
    //

    function blackoutScreen() {{
        if (!document.getElementById("blackout-overlay")) {{
            const overlay = document.createElement("div");
            overlay.id = "blackout-overlay";
            document.body.appendChild(overlay);
        }}
    }}

    function restoreScreen() {{
        const el = document.getElementById("blackout-overlay");
        if (el) el.remove();
    }}

    function installBlackoutListeners() {{
        document.addEventListener("visibilitychange", () => {{
            if (document.hidden) blackoutScreen();
            else restoreScreen();
        }});

        window.addEventListener("blur", blackoutScreen);
        window.addEventListener("focus", restoreScreen);
    }}

    // --- ВВОД ПАРОЛЯ ---

    function askPasswordAndDisable() {{
        const input = prompt("Введите пароль для отключения
защиты:");
        if (input === null) return;
        const real = xorDecrypt(ENC_PWD_B64, XOR_KEY);
        if (input === real) {{
            disableProtection();
            alert("Защита отключена.");
        }} else {{
            alert("Неверный пароль.");
        }}
    }}

    //
    // --- ИНИЦИАЛИЗАЦИЯ ---
    //

    document.addEventListener("DOMContentLoaded", () => {{
        enableProtection();
        installBlackoutListeners();
```

```
                document.addEventListener("keydown", (e) => {{
                    if (e.ctrlKey && e.shiftKey && e.key.toLowerCase() ===
"l") {{
                        e.preventDefault();
                        askPasswordAndDisable();
                    }}
                }});
            }});

        </script>
    </body>
    </html>
""")

# Далее создается файл, в который записывается код, указанный ранее
with open("protected.html", "w", encoding="utf-8") as f:
    f.write(html)

print("Файл protected.html создан.")
```

1.2.5 Тестирование скрипта

Проведем необходимые тесты. Для начала, откроем страницу (рисунок 7), и попробуем выделить текст на ней. Далее, попробуем убрать фокус со страницы (рисунок 8) и попробовать снять пароль (рисунки 9-11). После выполнения тестов сделаем вывод о работоспособности программы.

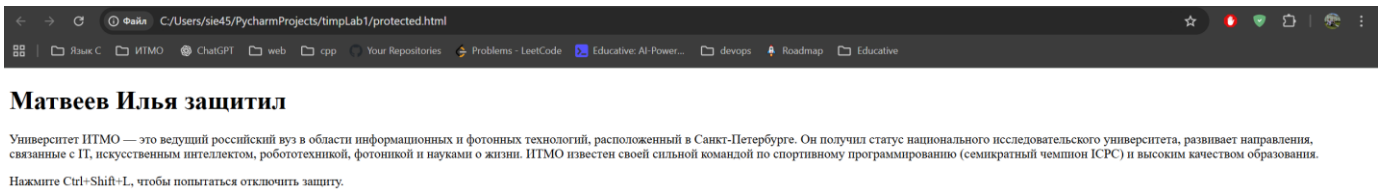


Рисунок 7 – Главная страница

Как мы видим, пока защита снята, возможно сделать скриншот программы. На самой странице мы видим заголовок и защищенный текст.

После включения защиты выделить текст становится невозможным, а любые горячие клавиши отказываются работать – иными словами, защита от копирования выполнена. Попробуем сделать скриншот страницы при помощь alt+prt.scr (не системным вызовом cntrl+shift+c), результат представлен на рисунке 2.

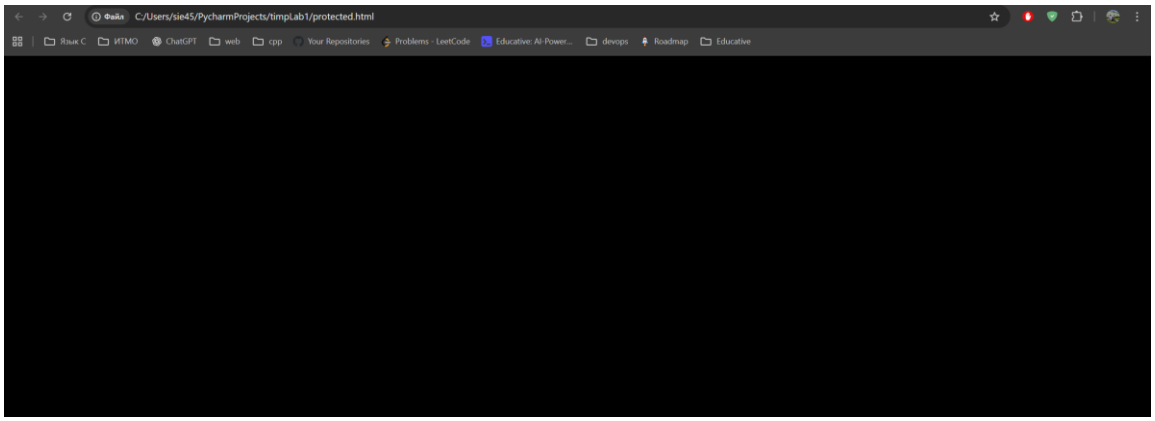


Рисунок 8 – Попытка скриншота

Как мы видим, при попытке скриншота стандартным методом страница затемняется. Однако это работает только на стандартный метод. Пользователь всё ещё может воспользоваться системным вызовом и сделать скриншот – обойти это выбранными средствами нельзя.

Далее попробуем снять защиту.

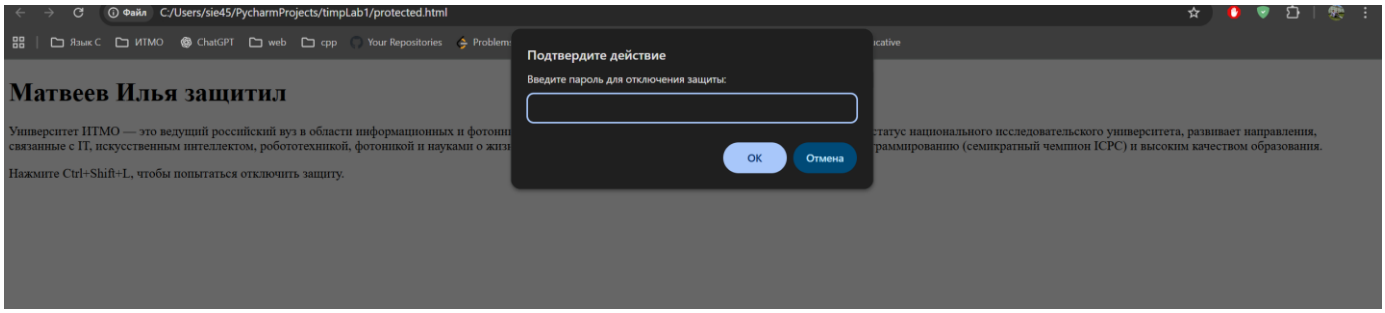


Рисунок 9 – Диалоговое окно с просьбой пароля

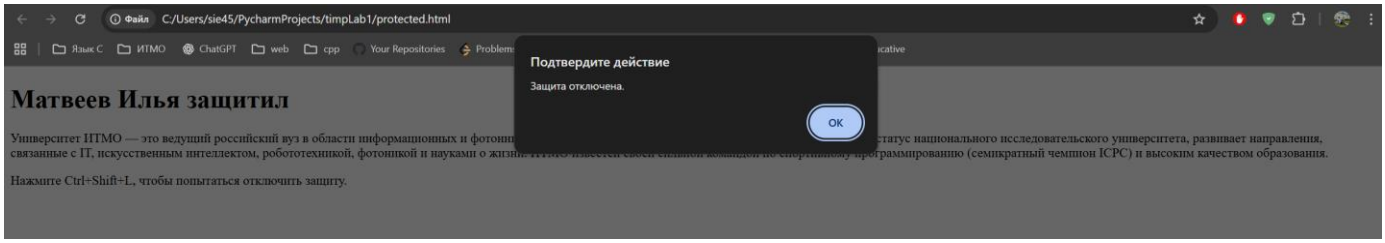


Рисунок 10 – Диалоговое окно с подтверждением

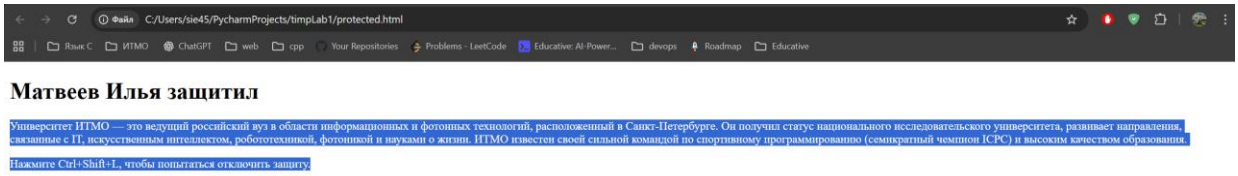


Рисунок 11 – Результат снятия защиты

Как видно из представленных рисунков, программа работает корректно – мы видим, что сначала требуется ввести пароль, после его ввода появляется подтверждение, и после этого мы снова можем копировать содержимое.

Иными словами, после проведения всех тестов мы видим, что все работает так, как указано в функционале.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были разработаны два программных решения, направленных на защиту информации на разных уровнях.

Создано десктопное приложение с графическим интерфейсом, которое обеспечивает избирательную защиту файлов в каталоге по заданным шаблонам имён с использованием системных механизмов ограничения доступа и неизменяемости. Реализована защита конфигурационного файла, хранение пароля в хэшированном виде и возможность управляемого включения и отключения режима защиты.

Разработан веб-скрипт, генерирующий HTML-страницу с комплексной клиентской защитой содержимого: запретом выделения и копирования текста, блокировкой контекстного меню и опасных сочетаний клавиш, автоматическим затемнением экрана при потере фокуса, а также возможностью отключения всех ограничений по паролю, хранящемуся в зашифрованном виде. При этом сохранена возможность печати страницы.

Выполнение работы позволило углубить практические навыки создания приложений с графическим интерфейсом, освоить методы управления правами доступа в операционной системе Linux, а также изучить современные приёмы клиентской защиты веб-контента от несанкционированного копирования и фиксации. Полученные решения демонстрируют работоспособность и соответствуют поставленным требованиям.